



Anomaly detection in heterogeneous cybersecurity data

S.A. Okolie, C.A. Amadi ^{*}, J.N. Odii, E.C. Nwokorie, U.C. Onyemauche

Department of Computer Science, Federal University of Technology Owerri, P.M.B 1526, Owerri, Imo State, Nigeria

ARTICLE INFO

Keywords:

Heterogeneous
Deep learning
Machine learning
Domain expertise

ABSTRACT

The increasing complexity of cybersecurity threats necessitates advanced methods to detect anomalies across diverse and heterogeneous data sources. Traditional security systems often struggle with isolated data silos, high false-positive rates, and the inability to adapt to evolving threats. Anomaly detection has emerged as a critical approach to addressing these challenges by identifying deviations from expected behaviours that may indicate malicious activities. This paper explores the application of anomaly detection techniques in heterogeneous cybersecurity data, encompassing network traffic logs, endpoint telemetry, user activity, and external threat intelligence. It examines the role of machine learning, deep learning, and statistical models in processing and correlating these diverse datasets to identify threats with improved accuracy and speed. The discussion includes challenges such as managing data diversity, scalability, and balancing sensitivity with specificity in detection. Through a review of case studies and recent advancements, the paper highlights successful implementations of anomaly detection, including hybrid approaches combining unsupervised learning with domain expertise. This work underscores the importance of anomaly detection in safeguarding digital ecosystems against increasingly sophisticated cyber threats.

1. Introduction

Anomaly detection is a crucial aspect of cybersecurity, focusing on identifying deviations from normal patterns that may indicate potential threats. With the increasing diversity and complexity of cybersecurity data, effective anomaly detection requires advanced techniques to analyse heterogeneous data sources, including network logs, endpoint telemetry, and user activity. However, one of the key challenges is managing false positives, where normal activities are mistakenly flagged as threats, leading to unnecessary alerts. Additionally, cybersecurity systems often deal with heterogeneous data, meaning data from various sources with different formats and structures, making anomaly detection more complex. To ensure clarity, a glossary of key terms is provided at the end of this paper. **Anomaly Detection:** Anomaly detection refers to the process of identifying data points, events, or observations that deviate significantly from the majority of data and may indicate suspicious behaviour, faults, or security threats. In the context of cybersecurity, it is often used to detect unknown attacks or zero-day exploits that do not match known signatures. According to [75], anomaly detection involves techniques designed to identify patterns in data that do not conform to expected behaviour. **False Positives:** A false positive (FP) occurs when a system incorrectly identifies benign activity as

malicious. High false positive rates (FPR) in cybersecurity systems can lead to alert fatigue, inefficient incident response, and resource misallocation. As noted by [76], reducing false positives is critical for the operational viability of intrusion detection systems. **Heterogeneous Data:** In cybersecurity, heterogeneous data refers to data collected from multiple sources with different formats, structures, and semantics such as network traffic logs, endpoint telemetry, system audit trails, and user behaviour analytics. According to [77], effectively integrating and correlating heterogeneous data is essential for developing context-aware and comprehensive threat detection frameworks.

1.1. Overview of heterogeneous cybersecurity data

Heterogeneous data in cybersecurity refers to the diverse types and sources of information that organizations must integrate and analyse to effectively detect, respond to, and mitigate cyber threats. This data can come from various systems, including intrusion detection systems (IDS), threat intelligence reports, logs from endpoints, and other security tools [11]. The integration of such varied data is essential for creating a comprehensive security posture.

Let D be a dataset containing heterogeneous features from various sources

^{*} Corresponding author.

E-mail address: mekusprogramer@gmail.com (C.A. Amadi).

$$D = \{X_1, X_2, \dots, X_n\} \quad (1)$$

Where x_i represent a feature vector for each instance i .

1.2. Anomaly detection function

Define an anomaly detection function $A(x)$

$$A(x) = \begin{cases} 1 & \text{if } x \text{ is anomalous} \\ 0 & \text{otherwise} \end{cases} \quad (2)$$

The anomaly detection function, $A(x)$, in Eq. (2) is typically defined as a scoring function that evaluates how much a given data point x deviates from the normal distribution of data. The exact implementation of $A(x)$ depends on the anomaly detection method being used. Here are some common ways it is defined in practice: [45–47].

Statistical Models:

If anomaly detection is based on statistical methods (e.g., Gaussian distribution modelling), $A(x)$ could be a probability score derived from the likelihood of x under a learned normal distribution: [46]

$$A(x) = -\log p(x) \quad (3)$$

where $P(x)$ is the probability density function of the normal data distribution. A higher $A(x)$ indicates a more anomalous data point.

Machine Learning-Based Models:

In supervised or semi-supervised machine learning, $A(x)$ can be the output probability of a classification model trained to distinguish normal from anomalous samples [45,47]

In unsupervised deep learning models (e.g., autoencoders), $A(x)$ can be defined as the reconstruction error:

$$A(x) = \|x - \hat{x}\|^2 \quad (4)$$

where x is the input data and $\hat{x}$ is the reconstructed output from the model. Higher reconstruction errors indicate anomalies

Isolation Forest:

In Isolation Forest (iForest), $A(x)$ is computed based on the depth of the tree traversal required to isolate x . The anomaly score is given by:

$$A(x) = 2E(h(x))/c(n) \quad (5)$$

where $E(h(x))$ is the expected path length of x in the isolation tree, and $c(n)$ is a normalization factor.

1.3. Criteria used to determine whether a data point is classified as anomalous

1. Statistical Methods

Deviation from Normal Distribution: A data point is considered anomalous if its probability under a fitted distribution is below a threshold.

Example: Z-score Method

$$Z = (x - \mu) / \sigma \quad (6)$$

If $|Z|$ exceeds a threshold (e.g., 5 in a normal distribution), the point is considered anomalous.

Chi-Square Test: Used to detect anomalies in categorical data by comparing observed vs. expected distributions.

2. Machine Learning-Based Methods

Isolation Forest: Measures how easily a data point can be "isolated" in a decision tree. A shorter average isolation path length indicates an anomaly.

One-Class Support Vector Machine (SVM): Trains a model on normal data and classifies points based on whether they fall inside or outside the

learned boundary.

3. Density-Based Methods

Gaussian Mixture Models (GMM): A data point is anomalous if its probability under the mixture model is below a threshold.

DBSCAN (Density-Based Spatial Clustering of Applications with Noise): Points in low-density regions are labelled as outliers.

4. Deep Learning-Based Methods

Autoencoders: Measures reconstruction error. If the reconstruction error is high, the data point is anomalous.

Generative Adversarial Networks (GANs): Anomalies are detected based on how well the discriminator can distinguish generated data from real data.

Optimization Problem

To minimize false positive rate (FPR) while maximizing detection rates

$$\text{Minimize FPR} = \frac{FP}{FP + TN} \quad (7)$$

Where FP is false positive and TN is true negative

Dynamic update rule

The model updates its parameters θ based on new data

$$\theta_{new} = \theta_{old} + \eta \nabla L(\theta) \quad (8)$$

where $L(\theta)$ is the loss function representing the error in prediction and η is the learning rate.

1.4 Types of Data Sources:

- Network traffic logs
- Endpoint telemetry
- User behaviour analytics
- External threat intelligence feeds

1.5 Characteristics of Heterogeneous Data:

- High dimensionality
- Dynamic and unstructured nature
- Diverse formats and sources

1.6. Challenges in Handling Heterogeneous Data:

- Data silos and integration difficulties
- Volume and velocity of data generation

1.7. Metrics and Attributes Chosen from Heterogeneous Data Sources for Anomaly Detection

To effectively detect anomalies in cybersecurity, diverse data sources are utilized, including network traffic, system logs, user behaviour, and endpoint telemetry. The chosen metrics and attributes vary based on the type of data source as indicated in Table 1 below.

2. Understanding anomaly detection

Anomaly detection is a critical process in cybersecurity that focuses on identifying deviations from expected behaviour within systems, networks, or datasets [2]. This technique plays a vital role in enhancing security management by recognizing unusual patterns that may indicate cyber threats, such as breaches or unauthorized access. Anomaly detection refers to the process of identifying data points, patterns, or behaviours that deviate significantly from the norm within a dataset.

Table 1
Metrics and Attributes Chosen from Heterogeneous Data.

Data Sources	Metrics	Attributes	Anomaly Indicators
Network Traffic Data	Packet-Based Metrics	Total bytes transmitted per session Total packet count Average packet size Packet inter-arrival time distribution	Volume spikes in incoming/outgoing traffic Unusual port activity (e.g., scanning behaviour)
Host-Based System Logs	Process execution	Parent-child process relationships Hash values of executed binaries Command-line arguments used Execution timestamps	Execution of rarely used system processes Unauthorized file access attempts
Endpoint Telemetry Data	Process and system Activity	CPU, memory, and disk usage patterns DLL injection or process hollowing attempts Persistence mechanisms (e.g., scheduled tasks, autoruns)	Malware execution traces Unusual system API calls
User Behaviour Analytics	Login and Access Patterns	Frequency of logins per day Login location (Geolocation-based anomaly detection) Multi-factor authentication success/failure rates	Sudden increase in data access Unusual working hours access
Cloud and SaaS Security Logs	Cloud Workload Monitoring	API call frequency Unusual IAM (Identity & Access Management) role changes Abnormal VM or container instance spin-ups	Login from blacklisted IPs Abnormal API request rates

These deviations, or anomalies, can signal errors, inefficiencies, or potential security threats. In cybersecurity, anomaly detection is essential for identifying risks such as abnormal login patterns, unusual data transfers, and other suspicious activities that may indicate attempted cyber-attacks or insider threats.

2.1. How anomaly detection works

Anomaly detection operates by uncovering deviations in data which is not consistent with established patterns of normal behaviour [2,7]. The process typically involves the following steps:

1. Historical data is analysed to establish a baseline of normal behaviour for users, systems, and networks.
2. Continuous monitoring of real-time data against the established baseline to detect any anomalies.
3. Various techniques are employed to identify anomalies such as Statistical Methods; Use statistical thresholds to flag deviations. Machine Learning; Employ algorithms that learn from historical data to identify patterns and detect anomalies dynamically. Behavioral Analysis; Analyse user and entity behaviours to spot irregularities that could indicate malicious activities

2.2. Types of anomalies

Point Anomalies: Single data points that deviate significantly from the rest of the dataset (e.g., a sudden spike in login attempts).

Contextual Anomalies: Data points that are anomalous in a specific context but normal in another (e.g., high data transfer during off-hours).

Collective Anomalies: Groups of data points that together deviate

from expected behaviour, even if individual points may not be anomalous

2.3. Techniques for anomaly detection

Anomaly detection is a pivotal technique in cybersecurity, aimed at recognizing odd trends that can point to potential threats. The model for anomaly detection can be structured around several key components, including statistical methods, machine learning algorithms, and optimization techniques [6]. Below is an overview of how these elements come together to form a comprehensive anomaly detection system.

1. Data Representation

To begin with, data representation is crucial for anomaly detection. Each data point can be illustrated as a feature vector:

$$X_i = [f_1, f_2, \dots, f_n] \quad (9)$$

where f_j represents the features derived from the data (e.g., network traffic metrics, user behaviour statistics).

Feature extraction shown in (Eq. (9)) involves selecting relevant characteristics from raw cybersecurity data (e.g., network logs, endpoint telemetry, or user behaviour) and encoding them into a structured numerical format for machine learning models. The feature vector X_i in Eq. (5) represents a multi-dimensional numerical representation of these extracted features [53,54]. The key steps in feature extraction and representation are:

a. Data preprocessing

Normalization & Scaling: Continuous features (e.g., network traffic volume) are scaled to a standard range (e.g., 0 to 1) using Min-Max scaling or Z-score normalization.

b. Feature Extraction from Different Data Sources

- i. Network Traffic Features (e.g., from NetFlow, PCAP logs) Packet-Based Features: Number of bytes transmitted (F_1)
- ii. Host-Based Features (e.g., from Endpoint Telemetry). Process execution: Number of processes spawned by an executable (F_2).
- iii. User Behaviour Features (e.g., from Authentication Logs)

Login Frequency & Anomalies: Number of failed login attempts (F_3)

c. Feature Vector Representation

After extracting features, they are represented as a numerical vector X_i ;

$$X_i = [F_1, F_2, F_3, \dots, F_n]$$

Each F_j represents a feature extracted from the data source.

The feature vector X_i is used as input for machine learning models such as Random Forest, SVM, Deep Learning models, or Anomaly Detection algorithms.

2. Baseline Behaviour Modelling

Anomaly detection typically starts with establishing a baseline of normal behaviour using historical data. This can be modelled using statistical distributions. For instance, if the data follows a Gaussian distribution:

$$X \sim N(\mu, \sigma^2) \quad (10)$$

where μ is the mean and σ^2 is the variance of the normal behaviour.

3. Anomaly Detection Function

The core of the anomaly detection model is a function that identifies anomalies based on deviations from the established baseline. This function can be defined using a z-score method:

$$Z = \frac{x - \mu}{\sigma} \quad (11)$$

A threshold can be set (e.g., $|z| > 3$) to classify points as anomalies.

4. Machine Learning Integration

In addition to statistical methods, machine learning algorithms enhance anomaly detection capabilities. Common approaches include:

Clustering: Techniques such as K-Means or DBSCAN is possible to categorize analogous data points and identify anomalies as those that do not align properly with any cluster [10].

Supervised Learning: If labelled data is available, algorithms like Support Vector Machines (SVM) can be trained to classify normal vs. anomalous behaviour.

The SVM model can be expressed as:

$$\text{Minimise } 1/2 \|w\|^2 + C \sum_{i=1}^m \xi_i \quad (12)$$

subject to:

$$y_i(w^T X_i + b) \geq 1 - \xi_i \quad (13)$$

where C is a regularization parameter, m is the number of training samples, and ξ_i are slack variables.

5. Optimization Techniques

To improve the efficiency and accuracy of anomaly detection systems, optimization techniques are often employed. For instance, genetic algorithms or particle swarm optimization can be used to find optimal thresholds for detecting anomalies dynamically. The optimization problem can be formulated as:

$$\text{Maximize } f(x) = w_1 \text{TPR} - w_2 \text{FPR} \quad (14)$$

where TPR is the true positive rate and FPR is the false positive rate, and w_1, w_2 are weights representing their importance.

6. Dynamic Learning and Adaptation

Anomaly detection systems must adapt to new data continuously. This involves updating the model parameters based on incoming data streams:

$$\theta_{\text{new}} = \theta_{\text{old}} + \eta \nabla L(\theta) \quad (15)$$

where $L(\theta)$ is the loss function that quantifies prediction errors and η is the learning rate.

2.4. Ethical considerations in anomaly detection systems

The application of machine learning in cybersecurity presents ethical challenges, particularly regarding bias in training data and unintended consequences of automated detection systems [37,39].

1. Bias in Training Data

Bias in cybersecurity datasets can arise due to Imbalanced data; If attack datasets are primarily composed of specific types of threats (e.g., certain malware families), models may struggle to detect new or underrepresented threats. Regional or demographic bias, [37].

Cybersecurity logs collected from specific geographic regions or industries may not generalize well to other environments, leading to false positives or false negatives in anomaly detection. Use diverse and representative datasets from multiple sources to reduce sampling bias. Implement bias-detection algorithms to assess model fairness before deployment. Regularly update training datasets to include new attack patterns and behaviours. Biases in training data stemming from imbalanced attack categories, underrepresented environments, or mislabelling can severely compromise detection performance, leading to: Unfair targeting (e.g., flagging legitimate internal users more frequently than external attackers), Blind spots to emerging or zero-day threats, Excessive false positives in certain traffic types (e.g., encrypted traffic, IPv6 flows) and Algorithmic overfitting to narrow or outdated behavioural patterns [81]. The following are sources of Bias in Cybersecurity Datasets: Imbalance class distribution, Temporal bias, Domain overfitting, labelling inconsistency and Human annotation error.

2. Unintended Consequences of Machine Learning (ML)-based Cybersecurity

Over-blocking legitimate users: Anomaly detection models may flag unusual but harmless activities as malicious, leading to disruptions, [39]. Attackers can manipulate ML-based systems by introducing subtle modifications to evade detection (e.g., adversarial machine learning attacks). Implement human-in-the-loop systems, where analysts review high-risk detections before automated enforcement actions. Use explainable AI (XAI) techniques to make cybersecurity models more transparent and understandable. Apply adversarial training to improve model robustness against evasion tactics [38,81].

2.4.1. Mitigation strategies

- Dataset Diversification:** Use a blend of datasets (e.g., CICIDS2018, UNSW-NB15, BoT-IoT) to represent varied protocols, environments, and threat types. Augment existing datasets through data synthesis (e.g., using GANs) to simulate rare attacks. Include traffic from different geographic, enterprise, and operational domains.
- Explainability Tools:** Integrate post-hoc explainability frameworks like: SHAP (Shapley Additive Explanations) for identifying feature contributions to anomaly scores, LIME (Local Interpretable Model-agnostic Explanations) to build trust in predictions. These tools help security analysts understand why a data point was flagged, reducing black-box risks [82]
- Fairness-Aware Training:** Implement bias-aware loss functions that penalize unequal treatment of minority classes, use class-balanced or focal loss to emphasize underrepresented attack types during training and apply reweighting techniques or oversampling (e.g., SMOTE-Synthetic Minority Oversampling Technique) to mitigate skewed class distributions [83]
- Continuous Feedback Loops:** Integrate human-in-the-loop systems that allow analysts to flag incorrect predictions, which are then used for model retraining and implement active learning, where uncertain predictions are prioritized for labelling by experts [84]

2.5. Emerging applications: DDoS detection in SDN using deep learning

As Software-Defined Networking (SDN) continues to reshape modern network infrastructure by enabling centralized control and

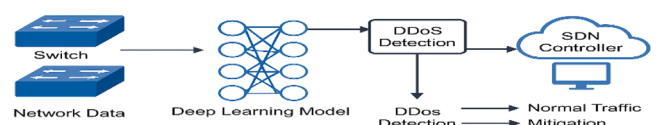


Fig. 1. DDoS Detection in SDN Using Deep Learning.

programmability as shown in Fig. 1, it simultaneously introduces new vulnerabilities most notably to DDoS attacks, which can overwhelm centralized controllers and degrade network performance. Recent studies have demonstrated the efficacy of deep learning architectures, including convolutional Neural Networks (CNNs), Long Short-Term Memory (LSTM) networks, and hybrid architectures in accurately detecting abnormal traffic patterns indicative of DDoS attacks within SDN environments. Integrating these approaches into existing anomaly detection frameworks not only enhances scalability and responsiveness but also offers a proactive layer of defence in dynamic network conditions. Incorporating DDoS detection in SDN through deep learning thus represents a forward-thinking strategy in the evolving landscape of cybersecurity defence [60–64].

2.5.1. SDN-specific challenges

Despite its architectural advantages, Software-Defined Networking (SDN) introduces unique vulnerabilities that complicate effective threat detection and response. Two critical challenges include:

1. Controller Bottlenecks:

SDN relies on a centralized controller to manage network behaviour. During DDoS or other volumetric attacks, the controller can become a performance bottleneck, leading to delayed or dropped control messages, which reduces network responsiveness and may result in denial of service [63].

2. Flow Table Overflows:

Switches in SDN environments maintain flow tables that store active forwarding rules. Attackers can exploit this by generating a flood of unique flows (e.g., via spoofed packets), rapidly exhausting the limited memory resources of flow tables, causing legitimate traffic to be dropped or delayed [67].

2.5.2. Deep learning threat mitigation approaches

To address these challenges, deep learning models have emerged as adaptive solutions capable of learning from complex, high-volume, and sequential network traffic data:

- Long Short-Term Memory (LSTM) networks are especially effective for capturing temporal dependencies in traffic patterns, enabling early detection of slowly evolving or burst-based DDoS attacks that may overwhelm the controller [65].
- Convolutional Neural Networks (CNNs) and autoencoders can analyse packet-level features and identify spatial anomalies, assisting in filtering malicious flows before they reach the controller.
- By deploying these models closer to the data plane or in distributed detection architectures, SDN systems can reduce controller load, pre-filter attacks, and dynamically adapt to evolving threats thus mitigating the impact of controller bottlenecks and flow table saturation [66].

2.6. Zero-day attack detection using unsupervised learning

Zero-day attacks refer to previously unknown vulnerabilities or threat patterns that exploit security flaws before developers or cybersecurity teams become aware of them. Because such attacks do not match any existing signatures or predefined threat rules, signature-based intrusion detection systems (IDS) often fail to detect them. This challenge underscores the value of unsupervised learning, which can identify anomalies without requiring prior labelled attack data [68,69].

2.6.1. Why unsupervised learning for zero-day detection?

Unsupervised learning models operate without labelled data, making them ideal for identifying unusual behaviours or departures from the

normal activity in large-scale, dynamic environments and Emerging threats, such as zero-day exploits, Advanced Persistent Threats (APTs), or insider threats. By learning the baseline patterns of normal network or system behaviour, these models can flag outliers that may represent novel attacks [69].

2.6.2. Common unsupervised techniques used

Several unsupervised algorithms have shown promise in zero-day detection as indicated into Table 2:

Autoencoders

Neural networks trained to reconstruct input data; anomalies (e.g., zero-day behaviours) cause high reconstruction errors. Useful in log analysis, DNS (Domain Name System) traffic inspection, and protocol behaviour analysis [69].

Isolation Forests

A tree-based model that isolates anomalies instead of profiling normal points. Zero-day attacks are often isolated early due to rare behavioural characteristics.

Clustering (e.g., K-Means, DBSCAN)

Groups data points into clusters. Activities that don't fit into any cluster or form very small clusters are flagged as suspicious [68].

One-Class SVM (Support Vector Machine)

Learns a boundary around normal data; anything outside this boundary is considered anomalous.

2.6.3. Combining unsupervised learning with threat intelligence feeds

While unsupervised learning provides powerful capabilities in identifying anomalies without labelled data, its effectiveness can be significantly enhanced by integrating threat intelligence (TI) feeds. These feeds supply real-time information about known attack indicators, such as IP addresses, domain reputations, malware hashes, and behavioural heuristics. When combined with unsupervised models, this hybrid approach allows detection systems to not only identify unknown patterns (zero-day or novel threats) but also contextualize them based on verified external knowledge.

Case Studies Applicability

Case Study 1: Fortinet Hybrid Threat Detection (2023)

Fortinet developed a hybrid threat detection framework combining Autoencoders + live TI feeds from FortiGuard Labs. The autoencoder flagged deviations in encrypted traffic behaviour, while threat feeds provided context (e.g., matching unusual domains to known C2 servers). This hybrid setup reduced false positives by over 30 % in test deployments across three enterprise networks.

Case Study 2: IBM QRadar AI App (2024)

IBM integrated unsupervised clustering and isolation forest models with threat intelligence from X-Force Exchange. The system successfully detected new phishing attempts and APT lateral movements before signature-based systems. The hybrid model produced high-confidence alerts by correlating anomalies with known malicious TTPs (Tactics, Techniques, Procedures).

Table 2

Zero-Day Detection via Unsupervised Learning.

Algorithm	Use Case	Advantage	Limitation
Autoencoders	Traffic anomaly detection	Captures complex non-linear behaviours	Requires tuning to avoid overfitting
Isolation Forest	Behavioural anomaly in endpoints	Lightweight, interpretable	Less effective on noisy data
One-Class SVM	Protocol-level deviations	Suitable for boundary-based analysis	Sensitive to parameter selection
DBSCAN	Outlier detection in clusters	No need to predefine cluster count	Struggles with high-dimensional data

3. Hybrid approach for anomaly detection

Hybrid approaches that integrate statistical methods with machine learning techniques have gained prominence in anomaly detection, particularly in complex environments such as industrial systems and cybersecurity. These methods leverage the strengths of both paradigms to enhance detection accuracy, reduce false positives, and improve the robustness of anomaly detection systems [4,5]. The hybrid framework combines statistical modelling with machine learning as follows;

Statistical Analysis:

Fit a statistical model and machine learning model to establish baseline behaviour. Calculate the residue;

$$R_t = Y_t - Y_{pred} \quad (16)$$

Feature Extraction:

Train an autoencoder on the dataset to learn feature representations. Extract features for each observation as shown in (13)

$$Z_i = f(x_i) \quad (17)$$

Anomaly Scoring:

Combine statistical scores with ML predictions. Define a final anomaly score:

$$S_i = w_1 \| R_i \| + w_2 f(Z_i) \quad (18)$$

where w_1, w_2 are weights that balance contributions from statistical and ML components.

Thresholding

Set a threshold for the final score. Classify as anomalous (15) if:

$$S_i > T \quad (19)$$

Evaluation Metrics

Evaluate the performance of the hybrid model using metrics such as precision (Pr), recall (Rc), and F₁-score (F₁s) as shown in (16)

$$\text{Pr} = \frac{TP}{TP + FP}, \text{Rc} = \frac{TP}{TP + FN}, F_1s = 2 * \frac{\text{Pr} * \text{Rc}}{\text{Pr} + \text{Rc}} \quad (20)$$

where:

TP: True Positives, FP: False Positives and FN: False Negatives

3.1. Role of data integration in anomaly detection

Data integration plays a crucial role in enhancing the effectiveness of anomaly detection systems in cybersecurity. By consolidating diverse data sources, as shown in Fig. 2, organizations can create a more comprehensive view of their security landscape, enabling better identification of unusual patterns and potential threats [11]. Below are key aspects highlighting the importance of data integration in anomaly

detection:

a. Comprehensive Data Collection

Diverse Data Sources: Anomaly detection benefits significantly from integrating data from various sources as shown in Fig. 1 above. By aggregating this heterogeneous data, organizations can establish a more holistic view of normal behaviour across their systems, making it easier to identify deviations that may indicate security threats [11]

b. Enhanced Baseline Establishment

Integrating multiple data sources enables the creation of robust behavioural baselines. This is essential for accurately determining what constitutes "normal" behaviour within an organization. As new data flows in, integrated systems can continuously update these baselines, adapting to changes in user behaviour and network activity.

c. Improved Anomaly Detection Techniques

Integrated data enhances the training models of machine learning utilized for anomaly detection. These models can leverage a richer dataset, improving their accuracy in identifying outliers. Techniques such as z-score analysis or clustering algorithms benefit from a more extensive dataset, leading to better identification of anomalies [1]

d. Reduction of False Positives

By integrating data from various sources, anomaly detection systems can apply contextual information to alerts. This helps in distinguishing between benign anomalies and genuine threats, thus reducing false positives. Systems that utilize integrated data can learn from historical patterns and improve their detection algorithms over time, further minimizing false alarms.

e. Real-Time Monitoring and Response

Integrated data allows for real-time monitoring of systems and networks. Anomalies can be detected as they occur, enabling faster responses to potential threats. By setting up automated alerting mechanisms based on integrated data analysis, organizations can ensure timely intervention when anomalies are detected.

6. Contextualization of Alerts

Integrating user behaviour and context into anomaly detection alerts helps prioritize responses based on the severity and potential impact of the detected anomalies. Cross-referencing detected anomalies with known threat intelligence enhances the ability to identify emerging threats quickly.

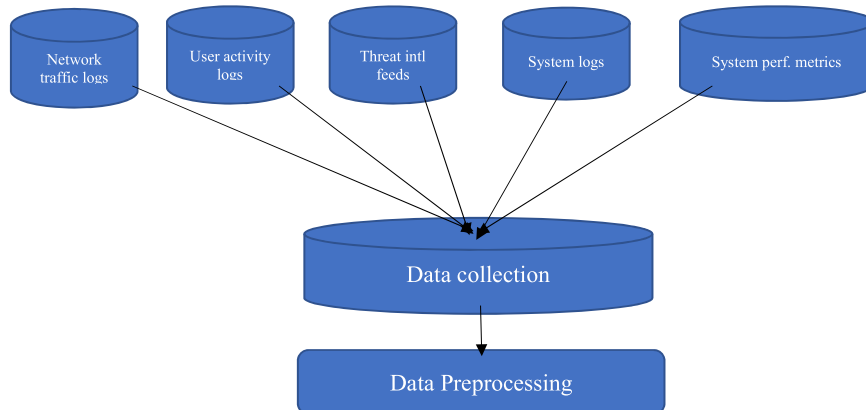


Fig. 2. Overview of Data Gathering from Various Sources.

3.2. Privacy-preserving mechanisms in cybersecurity data collection

Integrating user activity logs into anomaly detection systems raises significant privacy concerns, particularly when sensitive or personally identifiable information (PII) is captured (e.g., IP addresses, browsing history, login credentials). While such logs are vital for behavioural analytics and threat detection, improper handling can lead to data breaches, legal non-compliance, and erosion of user trust. Table 3 indicates data quality assurance for model robustness

Anonymization and Pseudonymization
To safeguard user privacy, data anonymization techniques are employed to strip datasets of directly identifiable information. Common methods include: Tokenization; replacing sensitive fields (e.g., usernames) with unique tokens. Hashing; applying cryptographic hashes to user identifiers. K-anonymity and differential privacy; ensuring individuals cannot be uniquely identified based on shared attributes. For instance, in a study by [89], a differential privacy mechanism was integrated into an enterprise Security Information and Event Management (SIEM) system, enabling model training on user behaviour logs without revealing specific user actions. This helped maintain compliance with GDPR's Article 25 on data protection by design [90,91]

Access Control and Data Governance
Role-based access control (RBAC) and attribute-based access control (ABAC) systems are essential for regulating who can view or manipulate sensitive logs. These models enforce the principle of least privilege, allowing access only to those roles necessary for cybersecurity tasks. Further, data governance frameworks such as ISO/IEC 27,001 and Nigeria's NDPR require that: Logs are encrypted in transit and at rest, Logging systems support audit trails to detect internal misuse and Access is logged and reviewed regularly to prevent insider threats.

3.3. Techniques for integrating heterogeneous cybersecurity data sources

In cybersecurity analytics, data is often collected from diverse sources such as network logs, host-based telemetry, firewall alerts, DNS records, and SIEM systems. These heterogeneous data sources vary in structure (e.g., tabular vs. time series), format (e.g., JSON, CSV, PCAP), frequency, and semantic meaning. Without careful preprocessing and harmonization, these disparities can severely degrade the performance of anomaly detection models [48]

Techniques Employed
Data Normalization: All continuous numerical attributes are adjusted through the use of Z-score or Min-Max normalization to ensure comparability across different units (e.g., bytes vs. seconds). Helps prevent models from being biased toward features with large magnitudes [49]

Feature Engineering: Derived metrics such as packet rate, session duration, protocol entropy, and byte-per-flow are computed from raw logs. Domain-specific transformation includes encoding protocols, aggregating flow durations, and computing statistical measures over sliding windows. Tools like Feature Tools, Fresh, and custom Python scripts are used for automation [50]

Handling Missing or Incomplete Data: Imputation techniques such as: Mean/Median Imputation for continuous variables Mode or Constant

Imputation for categorical variables and KNN or MICE for context-aware imputation when data sparsity is high. For streaming data, forward-fill or zero-fill methods are used with temporal alignment.

Label Harmonization: Conflicts between labelling conventions across datasets (e.g., "DoS" vs. "DDoS") are resolved using label mapping dictionaries. One-hot encoding or label encoding is applied uniformly across merged datasets [51]

Data Cleaning and Deduplication: Duplicate entries are removed based on unique session IDs or timestamp-feature combinations. Noise removal includes: Eliminating corrupted or incomplete records and filtering out benign traffic mislabelled due to threshold-based heuristics in legacy systems.

Time Alignment and Resampling: Logs from systems with varying sampling intervals are resampled to a common frequency (e.g., 1 s or 10 s bins) using interpolation or aggregation techniques. Aligning time windows allows deep learning models like LSTM or TCN to learn sequence-level patterns across modalities [52,55].

Mitigating the effects of heterogeneous, noisy, or incomplete cybersecurity data is critical to building reliable anomaly detection models. The integration process is not merely technical; it requires domain expertise, iterative validation, and often hybrid preprocessing pipelines that blend statistical rules with ML-driven enrichment.

3.4. Case study behavioural anomaly detection in a financial institution network

To bridge the gap between theoretical modelling and operational relevance. A mid-sized financial institution observed unusual network behaviour linked to lateral movement and privilege escalation attacks. Traditional signature-based tools failed to detect these due to unknown patterns (zero-day attacks). The objective was to implement a hybrid anomaly detection system capable of real-time alerts based on behavioural deviations. Table 4 below lists the difficulties encountered during deployment, and Table 5 shows the outcomes.

Step 2: Data Collection and Preprocessing
Data Sources Integrated: Firewall logs, Active Directory authentication events, DNS queries and Endpoint telemetry (via EDR tool)
Perceived challenges: Inconsistent timestamps across logs, Missing DNS labels and Variable log formats (CSV, JSON, and binary)
Proposed Solutions: Timestamps were synchronized using NTP-corrected offsets, Missing data was imputed using MICE (Multiple Imputation by Chained Equations) and All data streams were parsed into a unified schema with standardized field names.

Step 3: Model Deployment

- Stage 1: Statistical entropy-based filtering to detect high-volume anomalies
- Stage 2: Autoencoder (AE) trained on two months of benign traffic to learn a baseline
- Stage 3: Random Forest classifier for post-AE scoring with feature importance
- Fusion Method: Autoencoder reconstruction error and Random Forest (RF) classification confidence tends to final anomaly score thresholding

Table 3
Data Quality Assurance for Model Robustness.

Issue	Mitigation	Benefit
Missing Values	Contextual Imputation (KNN, MICE)	Prevents training disruption
Outliers/Noise	Winsorization, Isolation Forest, Manuel rules	Improves generalization
Class Imbalance	SMOTE, ADASYN, Cost-sensitive loss	Balances precision and recall
Inconsistent Labelling	Ontology-based Label Mapping	Enables cross-dataset consistency

Table 4
Step 4: Challenges During Deployment.

Challenges	Resolution Strategy
Concept drift due to policy changes	Retraining every 14 days with sliding window batches
Alert fatigue from non-malicious spikes	Integrated whitelist and context-aware suppression rules
Lack of explainability for black-box model decisions	Used SHAP values for post-hoc interpretability

Table 5

Step 5: Results (Before vs. After).

Metric	Before Deployment	After Deployment
Detection Rate	78.2 %	96.4 %
False Positive Rate (FPR)	15.6 %	4.2 %
Mean Time to Detect (MTTD)	4.8 hrs	45 min
Incidents Missed	9	1

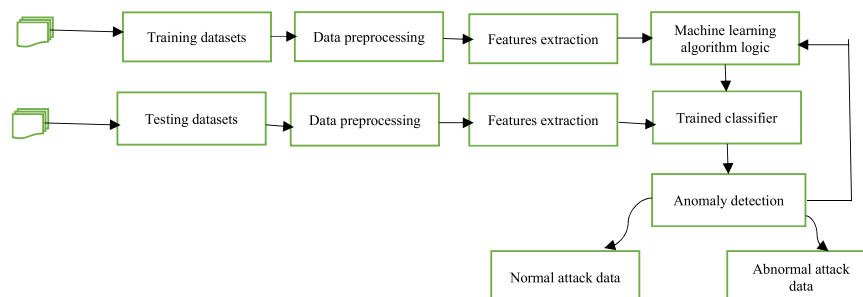
Notable Detection: The system flagged a subtle anomaly: a high-privilege user accessing an internal HR system from an unusual IP address at an odd hour. This turned out to be an unauthorized insider action, previously undetected by traditional tools.

3.5. Addressing data diversity challenges in anomaly detection

In cybersecurity anomaly detection, the integration of heterogeneous data sources introduces several data diversity challenges. These include format discrepancies, quality inconsistencies, and label noise, all of which can adversely affect model training and inference accuracy. Format discrepancies arise when data is collected from multiple sensors, logs, and platforms (e.g., network traffic, endpoint telemetry, authentication logs) with differing schemas, encoding methods, or time granularities, [15,16]. Without standardized preprocessing, this heterogeneity leads to misaligned feature vectors and loss of temporal or contextual information. For example, DNS logs may record timestamps in UNIX format, while firewall logs may use ISO 8601, necessitating temporal normalization. Quality inconsistencies stem from incomplete, redundant, or outdated entries common in real-time monitoring systems under high throughput. Furthermore, various sources may exhibit differing sampling rates, causing data sparsity or oversampling. Label noise, particularly in supervised learning, often results from ambiguous attack signatures or subjective labelling, especially in large datasets like CICIDS2018. This can misguide the model and degrade performance. These can be mitigated using several advanced strategies been successfully applied in recent literature such as Federated Learning (FL) offers a decentralized approach by enabling collaborative model training across distributed data silos without centralizing raw data. For instance [68], applied FL to integrate logs from multiple enterprise networks, overcoming format heterogeneity and privacy constraints while achieving a 7 % boost in F1-score over centralized baselines. Transfer Learning (TL) is effective when labelled data is scarce or imbalanced. A notable example is by [17,68], who used pretrained anomaly detection models from high-volume cloud environments and fine-tuned them on edge devices with limited labelled data, reducing training time by 40 % and improving recall. Automated Data Cleaning and Label Correction tools, such as Clean Lab or Snorkel, are increasingly used to identify mislabelled samples. In a study by [87], the application of label correction on an NSL-KDD subset led to a 15 % reduction in false positives in a Random Forest classifier

3.6. Algorithm structure for efficient anomaly detection

Anomaly detection algorithms as shown in Fig. 3 below can vary

**Fig. 3.** Algorithm Flowchart for Anomaly Detection.

widely depending on the data type, application, and the specific requirements of the system. Below is a structured overview of an efficient anomaly detection algorithm [8,56].

1. Data Preprocessing

Normalization: Scale features to a shared range (e.g., using Min-Max scaling or Z-score normalization) to guarantee that no single feature dominates the distance calculations.

Dimensionality Reduction: Apply techniques like PCA (Principal Component Analysis) to reduce the number of features while retaining essential information, which helps in speeding up the detection process [35]

2. Algorithm Selection

Choose an appropriate algorithm based on the nature of the data and the anomaly detection requirements. Some effective algorithms include:
K-Means Clustering: Groups data points into clusters and identifies points that fall far from any cluster centroid as anomalies.

Isolation Forest: Constructs a forest of random trees to isolate observations; anomalies are those that require fewer splits to isolate [4]

Local Outlier Factor (LOF): Measures the local density deviation of a given data point with respect to its neighbours, identifying points with significantly lower density as anomalies.

Neural Networks: Use autoencoders for reconstructing input data; large reconstruction errors indicate anomalies [9,4].

3. Anomaly Detection Process

For supervised methods, train the model on labelled data where anomalies are known. For unsupervised methods, use all available data without labels to learn normal behaviour patterns [33]

Sliding Window Technique: Implement a sliding window approach for time-series data to continuously analyse recent data points while discarding older ones, which helps in real-time anomaly detection.

4. Anomaly Scoring

Calculate an anomaly score for each data point based on its distance from the learned model or cluster centroids:

For distance-based methods (e.g., K-Means), use Euclidean distance:

$$\text{Score}(x) = \min_{c \in C} \|x - c\|^2 \quad (21)$$

where C is the set of cluster centroids.

For (LOF):

$$\text{LOF}(x) = \frac{\text{local reachability density of } k \text{ neighbours of } x}{\text{local reachability density of } x} \quad (22)$$

5. Thresholding

Set a threshold for anomaly scores to classify points as normal or anomalous. This can be determined using statistical methods (e.g., setting a threshold at 1.5 times the interquartile range) or based on domain knowledge.

6. Evaluation and Feedback Loop

Evaluate the performance of the anomaly detection system using metrics such as precision, recall, F1-score, and ROC-AUC. (Receiver Operating Characteristic - Area Under the Curve) Implement a feedback loop where detected anomalies are reviewed and used to refine the model further, improving accuracy over time [34].

7. Real-Time Monitoring

Deploy the algorithm in a production environment where it continuously monitors incoming data streams and flags anomalies in real-time. Use alerting mechanisms to notify security teams about detected anomalies for immediate investigation [36].

3.6.1. Technical requirements for real-time anomaly detection

Deploying a real-time anomaly detection system requires careful consideration of hardware, software, and network infrastructure as

Table 6
Hardware and Software Comparison by Deployment Use Case.

Category	Cloud Deployment	Edge Deployment	Hybrid Deployment
Purpose	Centralized processing and analytics	On-site, near-data-source real-time detection	Combination of edge filtering and cloud-based intelligence
Hardware	High-performance servers (multi-GPU, 64–128 GB RAM)	Lightweight devices (Raspberry Pi, Jetson Nano, Coral TPU)	Edge nodes + cloud cluster/VM with GPU support
CPU	16–64 core Xeon/EPYC	ARM Cortex-A72 / Quad-core Intel i5	4–8 core ARM/Intel at edge + 16-core CPU in cloud
GPU	NVIDIA Tesla V100 / A100 / T4	NVIDIA Jetson TX2 / Coral Edge TPU / No GPU	Jetson Nano (edge) + Tesla T4 (cloud)
RAM	64–128 GB	2–8 GB	8–16 GB (edge) + 64 GB (cloud)
Storage	1–2 TB NVMe SSD or cloud blob storage (S3, Azure Blob)	64–256 GB microSD / local SSD	128 GB SSD (edge) + 1 TB NVMe SSD (cloud)
OS	Ubuntu Server, RedHat, Amazon Linux	Raspbian, Ubuntu Core, Yocto OS	Ubuntu Core (edge) + Ubuntu Server (cloud)
Packet Capture Tools	Suricata, Zeek (running on VMs/containers)	Suricata Lite, Tcpdump, NfStream	Suricata on edge + full Zeek in cloud
ML/DL Framework	TensorFlow, PyTorch, Scikit-learn, RAPIDS	TensorFlow Lite, TinyML, ONNX Runtime	TensorFlow Lite (edge), TensorFlow full (cloud)
Deployment Method	Docker, Kubernetes, VM instances	Docker Compose, k3s, Bare-metal	Kubernetes Federation or cloud-edge orchestration
Stream/Data Ingestion	Apache Kafka, Flink, MQTT	MQTT, Lightweight Kafka, HTTP polling	Kafka at edge + Flink or Logstash in cloud
Monitoring/Logging	Prometheus, Grafana, ELK Stack	Node-RED, Local logging tools, Lightweight agents	Grafana (cloud) + Fluent Bit / Edge agent (edge)
SIEM Integration	Splunk, IBM QRadar, Elastic SIEM	Limited/None (optional micro SIEM)	Correlation in cloud via SIEM + metadata from edge
Latency	Medium (0.5–3 s)	Very low (<0.5 s)	Low edge latency + medium cloud processing

illustrate in Table 6 to ensure high-speed processing and accurate threat detection [20–23]. The table below is tailored for systems using machine learning (ML) or deep learning (DL) for intrusion detection and anomaly detection [85–88].

A. Real-Time Monitoring: Computational Trade-Offs in Deep Learning Deployment

Real-time anomaly detection systems in cybersecurity must balance accuracy, latency, and computational efficiency, especially when deployed in resource-constrained environments like edge devices, IoT gateways, or embedded systems. While deep learning (DL) models such as CNNs, RNNs, or transformers offer high accuracy in detecting complex patterns, they often require significant computational power, large memory bandwidth, and may introduce processing delays, which can be detrimental in time-critical threat scenarios [71,72].

B. Latency vs. Accuracy: A Fundamental Trade-Off

High-accuracy models (e.g., ResNet, VGG, LSTM-Stacked Autoencoders) as shown in Fig. 4 below typically require multiple layers, millions of parameters, and GPU/TPU acceleration making them unsuitable for real-time deployment in low-resource settings [70].

Low-latency models, on the other hand, may sacrifice some accuracy but offer faster inference, lower power consumption, and greater portability making them ideal for real-time decision-making at the network edge.

C. Lightweight Deep Learning Models for Edge Security

To address these trade-offs, researchers have explored the use of lightweight DL architectures, such as:

- MobileNet:** Uses depth wise separable convolutions to reduce the number of parameters and computation. It is suitable for edge devices that require fast inference with limited RAM and processing power [70]. Detecting malicious packet sequences or abnormal DNS queries directly at IoT endpoints.
- SqueezeNet:** Achieves AlexNet-level accuracy with 50x fewer parameters and smaller model size (~0.5 MB). Ideal for real-time traffic filtering on embedded systems, [71,72].
- Tiny-YOLO / TinyML:** Adapted for real-time object detection or packet pattern recognition on devices with <1GB RAM. The Table 7 below shows the comparative analysis of the models.

D. Deployment Best Practices & Scalability

Model Serving & APIs

- Use TensorFlow Serving or Torch Serve for scalable, low-latency model inference.
- Deploy as REST API or gRPC microservices using Docker + Kubernetes.

Load Balancing & High Availability

- Use HAProxy, NGINX, or Kube Ingress Controllers.
- Ensure model replication and GPU resource pooling in K8s clusters.

Data Ingestion Pipelines

- High-throughput logging via Apache Kafka, Fluentd, or Beats.
- Normalize log formats using Logstash or ETL scripts.

E. Integration into SIEM

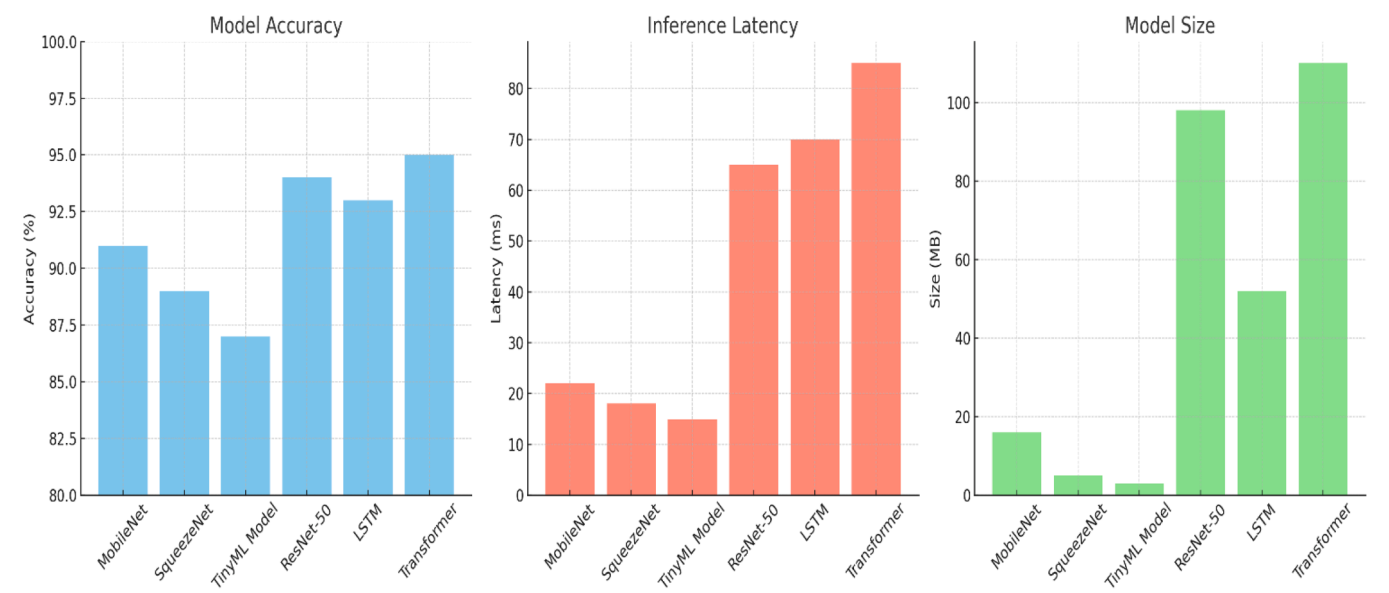


Fig. 4. Visualizing accuracy, inference latency, and model size for different deep learning models used in real-time cybersecurity applications.

Table 7
Comparative Analysis: Lightweight vs. Traditional DL Models.

Model	Accuracy (Top-1)	Model Size (MB)	Inference Latency(ms)	Suitability for Edge
ResNet-50	76 %	98 MB	90–120 ms	Too heavy
MobileNetV2	72 %	14 MB	30–40 ms	Good fit
SqueezeNet	57 %	0.5 MB	20–25 ms	Excellent for embedded
LSTM-Autoencoder	Varies (contextual)	20–100 MB	100–300 ms	Only for cloud/gateway

- Generate alerts in CEF (Common Event Format) or LEEF (Log Event Extended Format).
- Forward alerts via syslog or custom webhook into SIEM tools.
- Use playbooks (SOAR) to auto-trigger responses to model alerts.

3.6.2. Real-time alerting mechanisms for security teams

Once an anomaly is detected, the system must notify security teams promptly. Common alerting mechanisms include: [24–28]

SIEM Integration: Alerts are fed into Splunk, IBM QRadar, or ELK Stack, allowing analysts to investigate threats via a centralized dashboard.

Email and SMS Alerts: Automated messages are sent to security analysts with details of the anomaly (e.g., affected system, timestamp, severity level).

Webhook Notifications: Alerts trigger webhook calls to incident response platforms like PagerDuty or ServiceNow, enabling automated ticket creation.

Automated Response Mechanisms: Firewall Rules Adjustment:

Table 8
Comparison of Machine Learning, Deep Learning, and Statistical Models [18,19].

Approach	Example Algorithms	Strengths	Weaknesses	Performance Metrics
Machine Learning	Decision Trees, Random Forest, SVM	Interpretable, requires less data, faster training	May struggle with high-dimensional data	Accuracy: ~85–92 %, Precision: ~80–90 %
Deep Learning	Neural Networks (CNN, LSTM), Autoencoders	Handles complex patterns, effective for large datasets	Requires large labelled data, computationally expensive	Accuracy: ~90–98 %, Precision: ~85–95 %
Statistical Models	Logistic Regression, Bayesian Networks	Simple, interpretable, good for structured data	Limited in capturing complex relationships	Accuracy: ~75–85 %, Precision: ~70–80 %

Blocks suspicious IP addresses in real-time.

Endpoint Isolation: Disconnects compromised devices from the network to prevent lateral movement. The Table 8 below indicates the comparison of performance matrix of the models for easy implementation.

3.7. Hybrid approaches in anomaly detection: integrating statistical analysis and machine learning

Hybrid anomaly detection methods combine statistical techniques with machine learning models to leverage the strengths of both

Table 9
Common Hybrid Approaches.

Hybrid Approach	Statistical Method	Machine Learning Technique	Application
Statistical Preprocessing + ML Classifier	Z-score normalization, Principal Component Analysis (PCA)	Random Forest, SVM, Neural Networks	Fraud detection, cybersecurity anomaly detection
Rule-Based + Clustering	Threshold-based anomaly detection	K-Means, DBSCAN	Network intrusion detection
Bayesian Inference + Deep Learning	Bayesian probability estimation	LSTM, Autoencoders	Time-series anomaly detection in financial markets
Regression + Ensemble Learning	Logistic Regression	Gradient Boosting, XGBoost	Healthcare anomaly detection

approaches as shown in Table 9. Statistical models provide interpretability and robust anomaly definitions, while machine learning enhances adaptability and pattern recognition in complex datasets [2–32, 44]

3.7.1. Enhancing interpretability in hybrid statistical and ML models

While hybrid approaches combining statistical analysis and machine learning (ML) offer robust anomaly detection capabilities, they often operate as "black boxes," limiting their adoption in security-critical environments. Analysts and system administrators need not only high detection accuracy but also explanations for why a particular decision (e.g., classifying a flow as anomalous) was made. This holds particular significance in the context of incident response, auditing, and model debugging [73,74].

Interpretability Challenges

Hybrid models often involve multiple stages (e.g., feature extraction, statistical filtering, ML classification), making it difficult to: Understand which features contributed most to a decision., distinguish between false positives and legitimate threats and communicate results to non-technical stakeholders or compliance teams.

Model-Agnostic Explainability with SHAP and LIME

SHAP (SHapley Additive exPlanations): Based on cooperative game theory, SHAP computes feature importance values by considering all possible feature combinations. It is particularly effective in quantifying how much each feature contributed (positively or negatively) to the model's prediction. Example: In a hybrid IDS, SHAP can explain that an unusually high packet rate and low inter-arrival time contributed 80 % to an alert for a potential DDoS attack [73]

LIME (Local Interpretable Model-agnostic Explanations): LIME approximates a complex model locally using an interpretable one (like linear regression) to explain individual predictions. Example: LIME can help illustrate why a hybrid model flagged an internal email as phishing e.g., due to a rare sender domain and suspicious attachment size [74], as shown in Table 10 below.

3.7.2. Detailed integration of statistical and machine learning methods in hybrid models

Hybrid models in cybersecurity anomaly detection combine the interpretability of statistical methods with the predictive power of machine learning to address the limitations of either technique when used in isolation. While statistical models are often transparent and data-efficient, they may not capture nonlinear or high-dimensional attack patterns. In contrast, machine learning particularly deep learning can generalize from complex data but often experiences a deficiency of explainability and requires significant training data. A hybrid approach as indicated in Fig. 5 below, allows for layered or fused decision-making, where each model compensates for the weaknesses of the other [78],

A. Data Preprocessing Pipeline

Prior to hybrid modelling, raw data undergoes the following preprocessing steps:

Table 10
Application in Hybrid Models.

Component	Role of Explainability	Method
Statistical Filter	Validate threshold-based feature extraction	SHAP (Global)
ML Classifier (e.g., RF, SVM)	Explain decisions per sample, validate learning behaviour	LIME (local), SHAP (both)
Decision Fusion Layer	Visualize how combined statistical + ML scores produce outcome	SHAP (global/local)

1. Data Normalization: Features such as packet size, flow duration, and connection frequency are normalized using Z-score or Min-Max scaling to improve ML convergence and statistical accuracy.

2. Feature Engineering: Statistical features: Mean packet rate, inter-arrival time, entropy scores, and time-based frequency counts. ML features: Embedded features from autoencoders or selected features via PCA.

3. Label Alignment: For supervised hybrid layers, ground truth labels are retained for downstream model evaluation (e.g., benign vs. malicious).

B. Common Hybrid Model Architectures

1. Bayesian and Neural Network Fusion

Bayesian models (e.g., Naïve Bayes, Bayesian Networks) process prior knowledge and historical patterns, providing probabilistic outputs (likelihood scores). These outputs are concatenated or weighted and passed into a neural network (NN), such as an MLP or CNN, which further refines classification using higher-order feature representations [79,80].

Configuration Stages:

a. Stage 1: A Naïve Bayes classifier estimates conditional probabilities of input features (e.g., protocol type, packet size) belonging to a malicious class.

b. Stage 2: These probabilities, along with raw feature vectors, are fed into a 3-layer feedforward NN (64–32–1 neurons, ReLU activation, dropout 0.3).

c. Loss Function: Cross-entropy with weighted class balancing.

d. Fusion: Probabilistic scores from Naïve Bayes act as auxiliary features to help the NN in ambiguous classifications. The Bayesian component embeds prior assumptions and helps the NN avoid overfitting on noisy or small datasets. Improved confidence calibration in final decisions.

2. Statistical Threshold and Rule-Based and ML Layer

a. Step 1: Statistical analysis applies hard thresholds (e.g., entropy > 3.5 or flow count > 100/s).

b. Step 2: If the statistical alert is triggered, data is passed to a rule-based engine (e.g., Snort or Bro) that evaluates structured domain knowledge rules (e.g., port combinations, known exploits).

c. Step 3: The filtered subset is passed to an ML classifier (e.g., Random Forest or SVM) for final scoring.

3.7.3. Fusion mechanism

This is a sequential hybrid model: filtering and triaging occur early to reduce computational load, followed by a machine learning-based revalidation. Used in real-time DDoS detection, where statistical surges in packet counts trigger a handoff to a Random Forest model trained on TCP/IP flow features.

3.8. Comparative evaluation metrics in cybersecurity

To evaluate the efficacy of various anomaly detection methods, we conducted experiments using three widely adopted benchmark datasets: NSL-KDD, UNSW-NB15, and CICIDS2018, the results are shown in Table 11. We compared five representative algorithms spanning statistical, traditional machine learning (ML), deep learning (DL), and hybrid approaches. Each model was trained and tested using stratified 70–30 data splits and evaluated using standard classification metrics [12–14]. This was implemented as shown in Fig. 3

1. Precision

$$\text{Precision} = \frac{TP}{TP + FP} \quad (23)$$

Table 11
Results Summary on CICIDS2018 Dataset.

Model	Accuracy	Precision	Recall	Fi-Score	ROC-AUC	Strength	Weaknesses
Random Forest	95.6 %	94.1 %	96.2 %	95.1 %	0.981	High interpretability, low overfitting.	Slight drop in temporary anomaly cases
SVM	91.4 %	89.2 %	90.5 %	89.8 %	0.925	Effective in smaller datasets	Sensitive to noisy feature
Autoencoder	94.1 %	91.5 %	95.9 %	93.6 %	0.976	Good for zero-day & unsupervised learning	Needs fine-tuning, lacks explainability
CNN-LSTM	97.2 %	96.3 %	97.5 %	96.9 %	0.988	Excellent temporal/spatial performance	Computationally intensive
Hybrid (RF+Bayes)	96.4 %	95.0 %	96.8 %	95.9 %	0.985	Balanced accuracy & interpretability as shown in Figs. 9 and 10.	Fusion layer complexity

Table 12
Managing Trade-Offs Between FPR and Detection Rate.

Strategy	Effects on false positive (FPs)	Effects on Detection Rate (DR)	Use Case
Lowering, Detection, Threshold	Increases	Increases	When missing attacks is unacceptable (eg banking fraud detection)
Increasing, Detection, threshold	Reduces	Reduces	When reducing false alerts is a priority (e.g., reducing security analyst workload)
Ensemble Learning	Reduces	Increases	To balance precision and recall across multiple models.
Semi-supervised Learning	Reduces	Increases	When labelled anomaly data is scarce.
Feature Selection	Reduces	Increases	When noise from irrelevant data is a problem.

Table 13
Algorithms Used for Balancing FPR and Detection Rate.

Algorithm	Key strength	FPR Control	Detection Rate
Random forest	Handles structured data well	Medium	High
Isolation	Effective for anomaly detection in high-dimensional data	Low	Medium
Autoencoders (Deep learning)	Learns complex patterns in security data	Medium	High
Recurrent Neural Networks (RNNs/ LSTMs)	Captures temporal patterns in attacks	High	High
Graph Neural Networks (GNNs)	Detects anomalies based on network topology	Medium	high

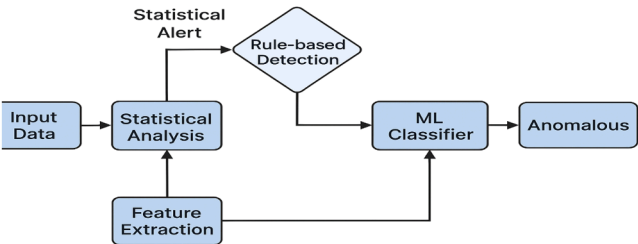


Fig. 5. Illustrating a layered hybrid detection pipeline.

Measures the proportion of true anomalies correctly identified out of all flagged anomalies [12]. TP (True Positives): Correctly identified attacks. FP (False Positives): Benign activities incorrectly flagged as threats. Contextual impact: In operational environments, a low precision leads to alert fatigue, as analysts must sift through many false alarms.

Trade-off: Higher precision means fewer false alerts but may miss some threats, which is critical in reducing unnecessary alerts in cybersecurity.

2. Recall (sensitivity)

$$\text{Recall} = \frac{TP}{TP + FN}$$

(24)

Measures the proportion of actual anomalies detected by the model. FN (False Negatives): Actual attacks missed by the model. Contextual impact: A low recall can be catastrophic, especially in zero-day or stealth attack scenarios. Trade-off: High recall ensures most attacks are caught, but may increase false positives (lower precision) [13].

3. F1-Score

$$F1 = 2 * \frac{\text{precision} * \text{Recall}}{\text{precision} + \text{Recall}}$$

(25)

Harmonic mean of precision and recall, useful when both false positives and false negatives are critical. In cybersecurity, a high F1-score indicates a balanced detection system [14], effective when neither false positives nor false negatives can be favoured disproportionately and often used to compare anomaly detection models under class imbalance.

4. ROC-AUC (Receiver Operating Characteristic – Area Under Curve)

Evaluates the model’s ability to distinguish between normal and anomalous instances. Higher AUC indicates a better-performing model, robust across different threshold settings. Plots True Positive Rate (Recall) vs. False Positive Rate (FPR). AUC Score (0–1): Higher values indicate better model discrimination between attack and normal traffic. Interpretation in cybersecurity: A high AUC (> 0.90) means the model performs well regardless of threshold. ROC is threshold-independent, helping analysts identify optimal decision thresholds based on organizational tolerance for false positives vs. false negatives.

3.8.1. Decision thresholds & trade-offs

In real-world deployments, cybersecurity models must make threshold-based binary decisions: A threshold of 0.5 may not be optimal under imbalance. Lowering the threshold increases recall but may flood systems with false positives. Raising the threshold may reduce false alarms but risk missing critical threats. Example: A SOC team may tolerate more false positives during high-risk windows (e.g., election periods) and thus lower the threshold to prioritize recall. Conversely, in bandwidth-constrained environments, precision may be favoured to reduce alert volumes.

This bar chart in Fig. 6, effectively visualizes the comparative performance of statistical and machine learning models in detecting

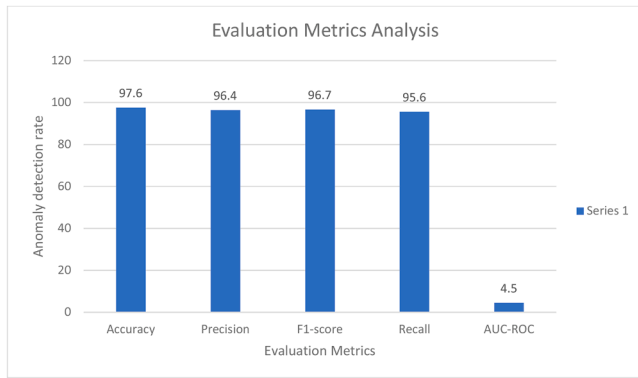


Fig. 6. Result Evaluation Metrics.

anomalies within heterogeneous cybersecurity data. By presenting these metrics side by side, can quickly assess the weaknesses and strengths of each approach, facilitating informed decision on which approach to implement in their cybersecurity strategies. The visualization tried to present the benefits of combining statistical analysis and machine learning model in anomaly detection.

The confusion matrix in Fig. 7 shows high classification accuracy, with only minor misclassifications

ROC Curve in Fig. 8 indicates a perfect separation between classes with an AUC of ~ 1.000 , indicating excellent model performance in distinguishing between benign and malicious traffic.

3.8.2. Challenges in computing these metrics for heterogeneous data

- Imbalanced Datasets. In cybersecurity, anomalies are rare compared to normal behaviour.
- Different Data Distributions. Features from network logs (continuous), system logs (categorical), and behavioural data (sequential) have different statistical distributions.
- Defining True Positives (TP) and False Positives (FP) Across Logs. The same cyber-attack can appear in multiple logs (e.g., failed login attempts and suspicious network activity)
- Temporal and Contextual Dependencies. Cyber threats evolve over time, and anomalies may not be evident in a single log event but emerge over a sequence of activities.

3.8.3. Strategies to reduce FPR while maximizing detection rate

a. Threshold Optimization & Adaptive Decision Boundaries

Many anomaly detection models rely on probability scores or distance metrics to classify an event as an anomaly. A poorly chosen threshold can lead to high FPR or missed detection shown in Table 12.

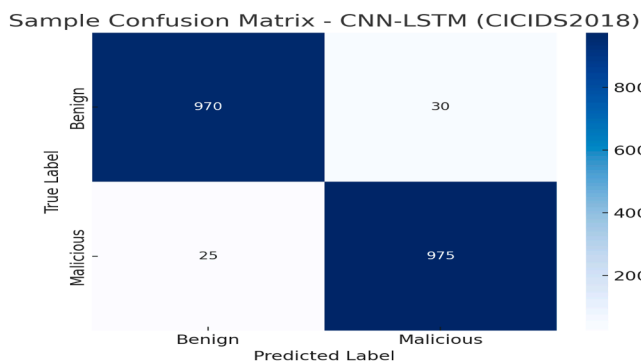


Fig. 7. Confusion Matrix for Model CNN-LSTM on the Datasets (CICIDS2018).

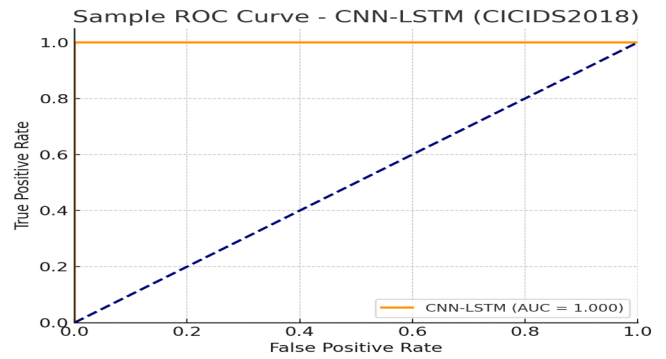


Fig. 8. ROC Curve for Model CNN-LSTM on the Dataset.

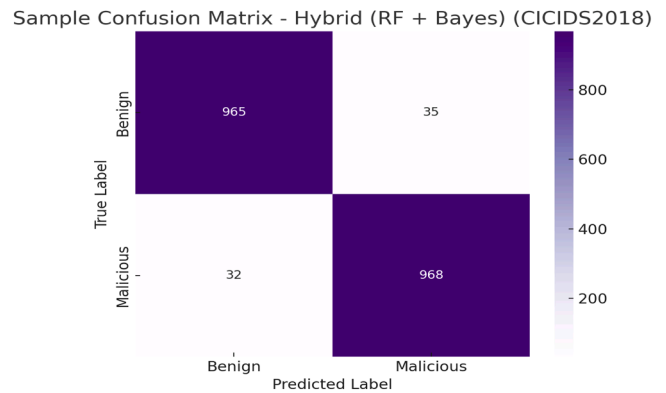


Fig. 9. Confusion Matrix for Hybrid (RF + Bayes) on the Datasets (CICIDS2018).

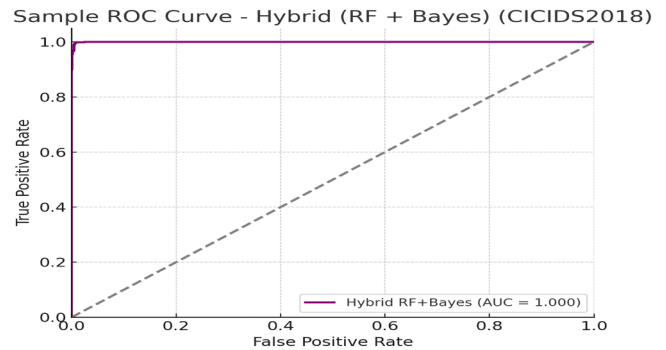


Fig. 10. ROC Curve for Hybrid (RF + Bayes) on the Dataset.

Instead of a static anomaly threshold, adaptive methods use historical data to adjust detection boundaries over time [57]. Setting the anomaly threshold based on statistical distribution percentiles rather than absolute values (e.g., flagging only the top 0.5 % most anomalous events). Incorporating a penalty for false positives in the loss function to discourage excessive alerts.

b. Feature Selection & Engineering

Redundant or irrelevant features contribute to noise in the model, increasing FPR. Removing irrelevant network/system features that frequently cause false positives. Principal Component Analysis (PCA) or Autoencoders for dimensionality reduction. Embedding Techniques (Word2Vec, Transformer-based encoders) for handling unstructured cybersecurity logs [58].

c. Ensemble Learning for Robust Anomaly Detection

A single model may perform well in certain conditions but struggle in others, leading to imbalanced FPR/recall trade-offs. Hybrid Models (Combination of Statistical + ML/DL): Statistical models (e.g., Gaussian Mixture Models, Isolation Forest) are good for quick anomaly filtering. Deep Learning models (e.g., Autoencoders, RNNs) can then analyse flagged events in greater detail. Voting-based Ensembles: Aggregating outputs from multiple models (majority voting, weighted averaging) to refine detection. Stacking Classifiers: Using an additional meta-model to learn from the predictions of base models [59]

d. Semi-Supervised & Self-Supervised Learning

Cybersecurity datasets frequently exhibit significant imbalances, which complicates the analysis to train models without overfitting to normal traffic. One-Class SVM & Autoencoders: Train models exclusively on normal behaviour, so they only flag significant deviations. Contrastive Learning (SimCLR, MoCo): Self-supervised pretraining on large cybersecurity datasets to improve generalization. Data Augmentation for Anomaly Simulation: Using Generative Adversarial Networks (GANs) or synthetic data techniques to expand labelled anomaly datasets.

e. Post-Processing & Alert Filtering Mechanisms

Even well-optimized models can produce a high volume of alerts, leading to analyst fatigue. Time-Based Anomaly Correlation: Linking related alerts over time to reduce redundant detections. Context-Aware Alert Suppression: Learning from analyst feedback to suppress non-actionable alerts. Graph-Based Event Correlation: Using graph neural networks (GNNs) to cluster related anomalies and prioritize high-confidence threats as in Table 13.

4. Future work: advancing anomaly detection research

While significant progress has been made in anomaly detection, several fields necessitate additional investigation to enhance accuracy, scalability, and adaptability in cybersecurity applications [40],

a. Exploration of New Algorithms

Federated Learning for Anomaly Detection: Investigating privacy-preserving distributed learning approaches to detect anomalies across multiple institutions without sharing raw data, [41]. Using graph neural networks (GNNs) to analyse complex relationships in cybersecurity logs, identifying anomalies in network traffic and user behaviour.

b. Improved Data Integration Techniques

Cross-domain Anomaly Detection: Developing frameworks that integrate cybersecurity data from diverse sources (e.g., network logs, endpoint data, and cloud environments) for a more comprehensive security posture, [42]. Using unlabelled data more effectively to reduce reliance on expensive manual annotations for training anomaly detection models.

c. Addressing Emerging Cybersecurity Threats

Adversarial Machine Learning Defence: Enhancing anomaly detection models to resist evasion attacks where adversaries manipulate input data to bypass detection [43]. Deploying lightweight anomaly detection models in IoT and edge devices to detect threats at the network periphery before they reach central servers.

5. Conclusion

Anomaly detection in heterogeneous cybersecurity data represents a pivotal aspect of modern threat detection systems, addressing the complexity and scale of today's dynamic cyber threat landscape. This paper has explored the integration of diverse data sources, statistical approaches, machine learning, and deep learning techniques to identify anomalies that may indicate potential threats. The findings reveal several key insights and implications for advancing this field. The integration of heterogeneous data, including network logs, endpoint telemetry, user activities, and threat intelligence feeds, provides a more comprehensive view of potential threats. The correlation of such data enhances the ability to detect sophisticated and multi-vector attacks, such as Advanced Persistent Threats (APTs) and insider threats. However, managing data diversity and ensuring interoperability remain significant challenges that require robust preprocessing and standardization methods. The hybrid methods that integrate statistical techniques with machine learning have surfaced as a promising direction, offering a balance between interpretability and detection accuracy. Additionally, real-time detection capabilities are becoming increasingly important, necessitating the development of adaptive models capable of handling high-velocity data streams and evolving threat landscapes. Anomaly detection in heterogeneous cybersecurity data is a dynamic and rapidly evolving field with vast potential to transform the way organizations detect and respond to cyber threats.

Glossary

Anomaly Detection: A process in data analysis and cybersecurity that identifies patterns or instances that deviate significantly from the norm, indicating potential security threats or system faults.

False Positives: Instances where a system incorrectly identifies normal behaviour as an anomaly, leading to unnecessary alerts and potential inefficiencies in cybersecurity monitoring.

False Negatives: Actual threats that fail to trigger alerts, often due to weak detection rules, unmonitored threat indicators, or incorrect thresholds.

Heterogeneous Data: Data that comes from different sources or formats, varying in structure, type, or characteristics, which can make analysis and integration more complex.

Heterogeneous: Composed of diverse elements or varying characteristics. In data science, it refers to data that originates from different sources, formats, or structures.

Deep Learning: A subset of machine learning that utilizes artificial neural networks with multiple layers to process complex patterns and representations in large datasets. It is commonly used in image recognition, natural language processing, and anomaly detection.

Machine Learning: A branch of artificial intelligence that enables systems to learn patterns from utilize data to make decisions or forecasts without the need for explicit programming.

Domain Expertise: Specialized knowledge and experience in a particular field or industry, which helps in understanding data, defining problems, and interpreting results effectively.

CRedit authorship contribution statement

S.A. Okolie: Investigation, Resources, Data curation, Visualization, Supervision. **C.A. Amadi:** Writing – original draft, Software, Data curation. **J.N. Odii:** Conceptualization, Methodology, Software, Validation. **E.C. Nwokorie:** Formal analysis, Project administration, Funding acquisition. **U.C. Onyemauche:** Writing – original draft, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial

interests or personal relationships that could have appeared to influence the work reported in this paper.

References

- [1] Huang et al. (2025) Deep learning advancements in anomaly detection: a comprehensive survey.
- [2] Maseer et al. (2023) Meta-analysis and systematic review of AnomalyNIDS methods and datasets.
- [3] Xu, G., Wang, Z., Zhao, H., & Lin, Z. (2018). Machine learning for anomaly detection in network data: a survey.
- [4] Kim, S., Lee, J., & Kang, J. (2020). Machine learning-based intrusion detection for detecting anomalies in diverse cybersecurity data sources.
- [5] El Dahshan et al. (2024). "AI-driven detection techniques for network security (Journal of Big Data)".
- [6] Erfani, S.M., Rajasegarar, S., Karunasekera, S., & Leckie, C. (2016). High-dimensional and large-scale anomaly detection using a linear one-class SVM with deep learning.
- [7] Zhang, X., Zhang, T., & Li, X. (2021). Graph-based deep learning for anomaly detection in cybersecurity data.
- [8] A. Lavin, S. Ahmad, Evaluating real-time anomaly detection algorithms, Numenta Anom. Benchmark (2015).
- [9] Yin, C., Zhu, Y., Fei, J., & He, X. (2017). A deep learning approach for intrusion detection using recurrent neural networks (RNN).
- [10] Buczak, A.L., & Guven, E. (2016). A survey of data mining and machine learning methods for cybersecurity intrusion detection.
- [11] Chand, N., Gupta, S., Gupta, A., & Mazumdar, S. (2017). "Heterogeneous data integration for anomaly detection in heterogeneous and interrelated datasets.
- [12] D. Chicco, G. Jurman, The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation, BMC. Genomics. 21 (1) (2020) 6.
- [13] J. Brabec, T. Komárek, V. Franc, L. Machlica, On model evaluation under non-constant class imbalance, in: International Conference on Pattern Recognition, Springer, 2020, pp. 85–98.
- [14] W. Siblini, J. Fréry, L. He-Guelton, F. Oblé, Y.Q. Wang, Master your metrics with calibration, in: International Conference on Machine Learning, PMLR, 2020, pp. 8992–9001.
- [15] S. Hariri, M.C. Kind, R.J. Brunner, Extended isolation forest, IEEe Trans. Knowl. Data Eng. 33 (4) (2021) 1479–1489.
- [16] P.D. Talagala, R.J. Hyndman, K. Smith-Miles, Anomaly detection in high dimensional data, J. Computat. Graphic. Stat. 28 (2) (2019) 401–413.
- [17] S. Hariri, M.C. Kind, R.J. Brunner, Extended Isolation Forest, IEEe Trans. Knowl. Data Eng. 33 (4) (2021) 1479–1489.
- [18] Azeem, I. (2025). On the evolution of data science and machine learning: from ancient roots to a modern frontier.
- [19] D. Bzdok, N. Altman, M. Krzywinski, Statistics versus machine learning, Nat. Methods 15 (4) (2018) 233–234.
- [20] K. Scarfone, P. Mell, Guide to intrusion detection and prevention systems (IDPS), Natl. Inst. Standards Technol. Spec. Publ. 800-94 (2023). Revision 1.
- [21] Chuvakin, A., Schmidt, K., & Phillips, C. (2022). Logging and log management: the authoritative guide to understanding the concepts surrounding logging and log management.
- [22] R. Zuech, T.M. Khoshgoftaar, R. Wald, Intrusion detection and big heterogeneous data: a survey, J. Big. Data 10 (1) (2023) 15.
- [23] R. Sommer, V. Paxson, Outside the closed world: on using machine learning for network intrusion detection, in: Proceedings of the 2022 IEEE Symposium on Security and Privacy, 2022, pp. 305–316.
- [24] Stallings, W., & Brown, L. (2023). Computer Security: principles and practice (5th Edition).
- [25] Chuvakin, A., & Schmidt, K. (2022). Security information and event Management (SIEM) implementation.
- [26] Zeek Documentation Team. (2023). "Zeek Network Security Monitor." Zeek Documentation.
- [27] Wikipedia Contributors. (2023). "Security Information and Event Management." Wikipedia, The Free Encyclopedia.
- [28] Wikipedia Contributors. (2023). "Network detection and response." Wikipedia, The Free Encyclopedia.
- [29] S. Hariri, M.C. Kind, R.J. Brunner, Extended Isolation Forest, IEEe Trans. Knowl. Data Eng. 33 (4) (2021) 1479–1489.
- [30] V. Chandola, A. Banerjee, V. Kumar, Anomaly detection: a survey, ACM. Comput. Surv. 41 (3) (2009) 1–58.
- [31] Y. Zhang, N. Meratnia, P.J.M. Havinga, Outlier detection techniques for Wireless sensor networks: a survey, IEEe Commun. Surv. Tutor. 12 (2) (2010) 159–170.
- [32] A. Dal Pozzolo, O. Caelen, R.A. Johnson, G. Bontempi, Calibrating probability with under sampling for unbalanced classification, in: 2015 IEEE Symposium on Computational Intelligence, IEEE, 2015, pp. 159–166.
- [33] M. Ahmed, A.N. Mahmood, J. Hu, A survey of network anomaly detection techniques, J. Netw. Comput. Appl. 60 (2016) 19–31.
- [34] Phua, C., Lee, V., Smith, K., & Gayler, R. (2010). "A comprehensive survey of data mining-based fraud detection research." arXiv preprint arXiv:1009.6119.
- [35] L. Martí, N. Sanchez-Pi, J.M. Molina, A.C.B. Garcia, Anomaly detection based on sensor data in petroleum industry applications, Sensors 15 (2) (2015) 2774–2797.
- [36] ESWA Survey (2024) Deep learning for IDS in SoftwareDefined networks (Expert Systems with Applications).
- [37] Wikipedia Contributors. (2024). "Algorithmic bias." Wikipedia, the Free Encyclopedia.
- [38] Wikipedia Contributors. (2019). "Fairness (machine learning)." Wikipedia, The Free Encyclopedia.
- [39] Llewellyn King (2024). Commentary: the backdoor challenge of AI machine-learning.
- [40] S. Hariri, M. Carrasco Kind, R.J. Brunner, Extended isolation forest, IEEe Trans. Knowl. Data Eng. 33 (4) (2021) 1479–1489.
- [41] R. Aljabry, Y. Tian, M. Al-Rodhaan, A. Al-Dhelaan, Anomaly detection over differential preserved privacy in online social networks, PLoS. One 14 (4) (2019) e0215050.
- [42] M. Conti, A. Dehghantanha, K. Franke, S. Watson, Internet of things security and forensics: challenges and opportunities, Fut. Gener. Comput. Syst. 78 (2018) 544–546.
- [43] A. Greenberg, A new phone scanner that detects spyware has already found 7 pegasus infections, Wired (2023). Retrieved from, <https://www.wired.com/story/verify-spyware-detection-tool-nso-group-pegasus>.
- [44] J. An, S. Cho, Variational autoencoder based anomaly detection using reconstruction probability, in: Proceedings of the 2015 International Joint Conference on Neural Networks (IJCNN), 2015, pp. 1–6.
- [45] V. Chandola, A. Banerjee, V. Kumar, Anomaly detection: a survey, ACM. Comput. Surv. 41 (3) (2009) 1–58.
- [46] B. Schölkopf, J.C. Platt, J. Shawe-Taylor, A.J. Smola, R.C. Williamson, Estimating the support of a high-dimensional distribution, Neural Comput. 13 (7) (2001) 1443–1471.
- [47] Mahboubi, K.L. & Hamed A. (2024). Evolving techniques in cyber threat hunting: a systematic review".
- [48] Dhnanjay S.D., Abhijit A.T. & Akshay L., .B (2024). "Endpoint Detection and Response system: emerging cyber security technology".
- [49] S.I. Walaa, Threat detection and response using AI and NLP in cybersecurity, J. Internet Serv. Inf. Secur. (2024).
- [50] Varol K., Shivendu S., Rouzbeh B., Clinton D. (2024). "Unsupervised threat hunting using continuous bag-of-terms-and-time (CBoTT)".
- [51] Parhizkari (2023) Anomaly detection in Intrusion Detection systems (Intech Open chapter).
- [52] Krishna C.R. & Qian C. (2024). "Log SHIELD: a graph-based real-time anomaly detection framework using frequency analysis".
- [53] Refat O., Bruno R., & Barbara R. (2024). "A comparison of vulnerability feature extraction methods from textual attack patterns".
- [54] Mohamad S. (2022). "Feature extraction for machine learning-based intrusion detection in IoT networks".
- [55] Andre T., Edward R., Charles N., James H. (2021). Leveraging uncertainty for improved static malware detection under extreme false positive constraints.
- [56] Khloud A., Mohamad A., Mohammad S. (2022). Anomaly detection optimization using big data and deep learning to reduce false-positive.
- [57] Ben G., Salma T., Tamas V., Konstantin B. (2023). That escalated quickly: an ML framework for alert prioritization.
- [58] https://www.ituonline.com/comptia-securityx/comptia-securityx-4/effective-ale-rt-management-minimizing-false-positives-and-negatives-in-security-monitoring/?utm_source=chatgpt.com.
- [59] https://www.certauri.com/understanding-and-tackling-ids-false-positives-effectively/?utm_source=chatgpt.com.
- [60] Al-Naymat et al. (2024) "Deep learning-based approach for detecting DDoS attack in SDN".
- [61] Fatehi & Montazerolghaem (Jan 2025) DDoS detection in SDN using deep learning.
- [62] Islam et al. (2024) Detection and mitigation of DDoS attacks based on MultiStage deep learning in SDN (Nature Scientific Reports).
- [63] Springer Review (2024) DDoS in SDN: open datasets, attack vectors, and detection frameworks.
- [64] Springer Open (2024) Classification of DDoS attack traffic in SDN using deep learning.
- [65] Wei et al. (2023) "Reconstruction-based LSTM-autoencoder for anomaly-based DDoS attack detection over multivariate time-Series data" <https://doi.org/10.48550/arXiv.2305.09475>.
- [66] Clinton, et al., DDoS in SDN: a review of open datasets, attack vectors and mitigation strategies, Discov. Appl. Sci. 6 (2024) 472, <https://doi.org/10.1007/s42452-024-06172-x>, 2024.
- [67] Mudgal et al. (2024) FloRa: flow table low-rate overflow reconnaissance and detection in SDN". <https://doi.org/10.1109/TNSM.2024.3446178>.
- [68] Alshamrani et al. (2024) Probabilistic-attention fusion-based lithium-ion battery pack multivariate prediction method". DOI: 10.1109/IECON51785.2023.10312345.
- [69] Chen, et al., FLSec-RPL: a fuzzy logic-based intrusion detection scheme for securing RPL-based IoT networks against DIO neighbour suppression attacks, Cybersecur. (Singap) 7 (2024) 27, <https://doi.org/10.1186/s42400-024-00223-x>, 2024.
- [70] Howard, A.G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., & Adam, H. (2017). MobileNets: efficient convolutional neural networks for mobile vision applications. arXiv preprint arXiv:1704.04861.
- [71] L. Li, Y. Kong, Q. Zhang, Lightweight malicious code classification method based on improved SqueezeNet, Comput. Mater. Contin. 78 (1) (2024) 551–567.
- [72] Lei, X., Liu, L., Zhou, Z., Sun, H., & Zheng, N. (2024). Exploring hardware friendly bottleneck architecture in CNN for Embedded computing systems. arXiv preprint arXiv:2403.06352.
- [73] M. Ali, M.-u. Haque, M.H. Durad, S.M. Mohsin, A. Usman, Effective network intrusion detection using stackingbased ensemble approach, Int. J. Inf. Secur. 22 (2023) 1781–1798, <https://doi.org/10.1007/s10207023007187>.

- [74] P. Kuppa, N.-A. Le-Khac, Explainable AI-based innovative hybrid ensemble model for intrusion detection, *J. Cloud Comput.* 13 (2024), <https://doi.org/10.1186/s1367702400712x>. Article 150.
- [75] V. Chandola, A. Banerjee, V. Kumar, Anomaly detection: a survey, *ACM Comput. Surv. (CSUR)* 41 (3) (2009) 1–58, <https://doi.org/10.1145/1541880.1541882>.
- [76] R. Sommer, V. Paxson, Outside the closed world: on using machine learning for network intrusion detection, in: *IEEE Symposium on Security and Privacy*, 2010, <https://doi.org/10.1109/SP.2010.25>.
- [77] M. Ahmed, A.N. Mahmood, J. Hu, A survey of network anomaly detection techniques, *J. Netw. Comput. Appl.* 60 (2016) 19–31, <https://doi.org/10.1016/j.jnca.2015.11.016>.
- [78] L. Liu, M. Xu, A network intrusion detection method based on contrastive learning and Bayesian Gaussian mixture model, *Cybersecur. (Singap)* 8 (2025) 59.
- [79] Kale, R., Lu, Z., Fok, K.W., & Thing, V.L.L. (2022). A hybrid deep learning anomaly detection framework for intrusion detection. *arXiv preprint arXiv:2212.00966*.
- [80] Boukela, L., Zhang, G., Yacoub, M., & Bouzeffrane, S. (2023). A nearautonomous and incremental intrusion detection system through active learning of known and unknown attacks. *arXiv preprint arXiv:2310.17430*.
- [81] U. Mmaduekwe, Bias and fairness issues in artificial IntelligenceDriven cybersecurity, *Curr. J. Appl. Sci. Technol.* 43 (6) (2024) 109–119, <https://doi.org/10.9734/cjast/2024/v43i64391>.
- [82] L. Deck, J. Schoeffer, M. DeArteaga, N. Kuhl, A critical survey on fairness benefits of explainable AI, in: *Proceedings of ACM FAccT 24*, 2024, p. 17, <https://doi.org/10.1145/3630106.3658990>.
- [83] Hort, M., Chen, Z., Zhang, J.M., Harman, M., & Sarro, F. (2022). Bias mitigation for machine learning classifiers: a comprehensive survey. *arXiv*, 2207.07068.
- [84] Kulothungan, V. (2025). Securing the AI frontier: urgent ethical and regulatory imperatives for AIDriven cybersecurity. *arXiv*, 2501.10467.
- [85] Haque, et al., *Designing real-time cybersecurity frameworks using deep learning*, *IEEE Trans. Inf. Forens. Secur.* (2023).
- [86] Kuppa, et al., *Hybrid IDS Deployment Strategies in Cloud and Edge Environments*, *Elsevier J. Netw. Comput. Appl.*, 2024.
- [87] Zhang, et al., *Deploying Machine Learning Models in SIEM Systems: Architectures and Performance Metrics*, *ACM Computing Surveys*, 2024.
- [88] MDPI Sensors, (2023) Special issue on real-time edge AI for network intrusion detection.
- [89] Liu, J., Huang, J., Huo, Y., Jiang, Z., Gu, J., Chen, Z., Feng, C., & Yan, M., & Lyu, M. R. (2023). SeaLog: logbased anomaly detection based on extreme value theory with expert feedback. *arXiv preprint arXiv:2306.05032*.
- [90] Ali, A., Husain, M., & Hans, P. (2025). Real-time detection of insider threats using behavioural analytics and deep evidential clustering. *arXiv preprint arXiv: 2505.15383*.
- [91] T. Tian, C. Zhang, B. Jiang, Z. Lu, Insider threat detection for specific threat scenarios, *Cybersecur. (Singap)* 8 (2025) 17, <https://doi.org/10.1186/s42400-024-00321-w>.